

Proyecto de corte — Arquitectura de Software

Sistema de diagnóstico del estado de plantas (base de la *matera inteligente*) · Semestre 2026-03

1. Contexto

El producto final que este curso construye por etapas es una **matera inteligente**: un dispositivo basado en ESP32 que mide condiciones ambientales de una planta, las envía a un servicio en la nube y ofrece al usuario un acompañamiento —a través de una aplicación móvil y un front web— para mantener la planta sana. En etapas posteriores el producto incorporará gestión de usuarios, múltiples plantas por usuario, histórico de mediciones y una capa de gamificación que convierte el cuidado de la planta en un juego.

Nada de eso se construye en este corte. Lo que se construye es el **núcleo**: el servicio que, dadas unas mediciones y una especie, determina el estado de la planta. La razón de empezar por ahí es arquitectónica, no funcional: si ese núcleo queda bien separado de *cómo llegan los datos* y de *dónde vive la tabla de referencia*, todo lo demás entra después sin reescribirlo. Si queda mal separado, cada requerimiento nuevo obligará a tocar la lógica de negocio, y ese es exactamente el escenario que el curso pretende enseñarle a evitar.

La pregunta que este proyecto evalúa no es "¿funciona?", sino: *cuando llegue el ESP32 enviando datos por MQTT, cuando la tabla de referencia pase de un CSV a una base de datos, y cuando aparezcan usuarios y gamificación, ¿cuánto de lo que escribiste hoy tendrás que cambiar?* La respuesta correcta es: la capa de dominio, cero.

2. Alcance de este corte

Sí entra	No entra
• API web que expone el diagnóstico como servicio • Lógica de validación del estado de la planta • Tabla de referencia por especie (CSV o base de datos) • Front web separado que consume la API de forma asíncrona • Pruebas unitarias de la lógica de negocio • Documento de arquitectura	• Hardware, ESP32, sensores reales • Autenticación y gestión de usuarios • Persistencia de mediciones históricas • Gamificación • Aplicación móvil • Despliegue en nube

Los elementos de la columna derecha no se implementan, pero **sí se diseñan en el papel**: la sección 6.b del entregable exige explicar dónde entraría cada uno sin romper el diseño actual.

3. Requisitos funcionales

ID	Requisito
RF1	El sistema recibe una medición compuesta por humedad (% de humedad del sustrato), nivel de luz (lux) y temperatura (°C), junto con la especie de la planta.
RF2	El sistema compara cada parámetro contra los rangos de referencia de esa especie y clasifica cada uno como BAJO, OPTIMO o ALTO.
RF3	El sistema deriva un estado global de la planta a partir de los estados individuales, según una regla de agregación que el equipo define y justifica (p. ej. SALUDABLE / EN_RIESGO / CRITICO).
RF4	El sistema devuelve, junto al estado, una recomendación textual por cada parámetro fuera de rango.
RF5	El sistema expone la lista de especies soportadas con sus rangos de referencia.
RF6	El sistema maneja explícitamente los casos de error: especie desconocida, parámetro ausente, valor no numérico y valor fuera del rango físicamente posible. Cada caso responde con un código HTTP adecuado y un cuerpo de error uniforme.

La tabla de referencia mínima debe cubrir **al menos cinco especies**. Los valores deben ser plausibles y el equipo debe citar la fuente de donde los tomó.

4. Restricciones de arquitectura

Estas restricciones no son negociables y son el objeto principal de la evaluación.

ID	Restricción
RA1	El backend no genera HTML. Ni plantillas del lado del servidor (Jinja, Thymeleaf, JSP, Blade), ni <i>strings</i> de HTML contruidos en el controlador, ni renderizado de vistas. El backend responde exclusivamente con JSON. Lo que se está construyendo es una API, no un sitio.

RA2	El front es un cliente independiente (HTML, CSS y JavaScript estáticos), servido por separado del backend, que consume la API mediante peticiones asíncronas (fetch / async await). La página no se recarga al consultar un diagnóstico.
RA3	El backend está organizado en capas con responsabilidades separadas : presentación (controladores REST y DTO), aplicación (casos de uso y orquestación), dominio (entidades y reglas de negocio) e infraestructura (acceso a la tabla de referencia, configuración, framework). La separación debe ser visible en la estructura de paquetes o módulos.
RA4	La lógica de validación es agnóstica a la obtención de los datos. Ninguna clase o módulo del dominio importa nada del framework web (flask, org.springframework.*), ni del mecanismo de persistencia (csv, pandas, JDBC, JPA), ni de la biblioteca HTTP. Criterio de verificación: el dominio debe poder ejecutarse desde una prueba unitaria <i>sin levantar el servidor y sin leer el archivo o la base de datos</i> .
RA5	El acceso a la tabla de referencia se hace a través de una abstracción declarada en el dominio (interfaz / puerto / protocolo) e implementada en infraestructura. La dirección de la dependencia va de infraestructura hacia el dominio, nunca al revés. Cambiar el CSV por una base de datos debe implicar una clase nueva y cero cambios en el dominio .
RA6	La entrada de la petición se valida y se transforma en objetos del dominio en el borde del sistema. El dominio nunca recibe un request, un dict crudo ni un Map<String, Object>.
RA7	CORS configurado para permitir que el front, servido desde otro origen, consuma la API.
RA8	Stack: Python con Flask o Java con Spring Boot . Un solo lenguaje por equipo.

Descalificación parcial. El incumplimiento de **RA1** o de **RA4** lleva a cero el componente "Cumplimiento de restricciones" de la rúbrica, sin importar la calidad del resto del trabajo. Son las dos restricciones que definen el ejercicio.

5. Principios SOLID

El documento de arquitectura debe demostrar, para cada uno de los cinco principios, dónde se aplicó y qué habría pasado de no aplicarlo, citando **archivo y línea** del repositorio propio. Como orientación de dónde suele aparecer cada uno en este problema:

- **SRP** — la clase que decide el estado de la planta no es la misma que lee el CSV ni la que arma la respuesta HTTP.
- **OCP** — agregar un cuarto parámetro (por ejemplo pH del sustrato) o una nueva regla de agregación no debería obligar a modificar el evaluador existente.
- **LSP** — cualquier implementación de la abstracción de RA5 debe ser sustituible sin que el dominio se entere.
- **ISP** — el dominio depende de la operación que necesita ("dame los rangos de esta especie"), no de una interfaz de repositorio genérica con doce métodos.
- **DIP** — es la formalización de RA5.

Se valora más una aplicación honesta con una tensión reconocida ("aquí violamos parcialmente ISP y esta fue la razón") que una lista de cinco afirmaciones genéricas.

6. Entregables

a. Repositorio

Repositorio Git público o con acceso para el docente, con historial **incremental**. Un repositorio con un único *commit* inicial se califica como entrega sin proceso y pierde el componente correspondiente. Debe incluir un README con instrucciones de ejecución del backend y del front, y las pruebas unitarias del dominio.

b. Documento de arquitectura (máximo 8 páginas)

1. **Diagrama de paquetes / componentes** que corresponda *uno a uno* con la estructura real del repositorio. Un diagrama que no coincida con el código se califica como no entregado.
2. **Diagrama de secuencia** del recorrido completo de una petición de diagnóstico, desde el fetch del front hasta la respuesta, nombrando las clases o módulos reales que participan.
3. **Tabla de responsabilidades por capa**: qué hace cada capa, qué tiene prohibido hacer, y de qué depende.
4. **Justificación SOLID** según la sección 5, con referencias a archivo y línea.
5. **Plan de evolución** (máximo 2 páginas): para cada uno de los cuatro escenarios siguientes, qué componentes se agregan, cuáles se modifican y cuáles no se tocan:
 - las mediciones dejan de llegar por HTTP y llegan por MQTT desde un ESP32;
 - la tabla de referencia migra de CSV a una base de datos relacional;
 - aparecen usuarios, cada uno con varias plantas;
 - se agrega gamificación (puntos, rachas, niveles) sobre el cuidado de la planta.
6. **Decisiones y alternativas descartadas**: al menos tres decisiones de diseño, cada una con la alternativa que se consideró y la razón del descarte.

c. Front web funcional

Formulario con los tres parámetros y el selector de especie —poblado desde **RF5**, no escrito a mano en el HTML—, consulta asíncrona y presentación del diagnóstico, incluyendo el manejo visible de los errores de **RF6**.

d. Pruebas unitarias del dominio

Mínimo seis pruebas que ejerciten las reglas de negocio **sin** levantar el servidor web y **sin** leer el archivo o la base de datos reales (usando dobles de prueba de la abstracción de **RA5**). Estas pruebas son la evidencia ejecutable de **RA4**.

e. Bitácora de uso de IA (máximo 1 página) — suma hasta 5 %

Relato breve y honesto del trabajo con asistentes de IA: qué se le pidió, qué propuso, qué se aceptó tal cual, qué se cambió y **por qué** se cambió. No se penaliza ningún nivel de uso; se premia la lucidez sobre el propio proceso. Un párrafo del tipo "nos generó los controladores llamando directamente al lector de CSV; lo separamos porque violaba RA4, y así fue como entendimos DIP" vale más que las tres páginas siguientes del documento.

7. Sustentación

Cada equipo sustenta ante el docente durante **8 a 10 minutos**. No hay presentación preparada ni diapositivas: se responden preguntas sorteadas de un banco, sobre el propio código y el propio documento, con el repositorio abierto en pantalla. **Todos los integrantes responden**; la pregunta se asigna a un integrante específico y no puede ser respondida por otro.

Las preguntas son *de mutación*: plantean un cambio hipotético y piden identificar su impacto. Ejemplos del tipo de pregunta que se hará:

- Mañana el dato llega por MQTT desde un ESP32 en lugar de por HTTP. ¿Qué archivos de tu repositorio cambias y cuáles garantizas que no se tocan? Ábrelos.
- La tabla de referencia pasa de CSV a PostgreSQL. Muéstrame la clase que tendrías que escribir y la línea exacta donde se conecta con el resto del sistema.
- Se agrega un cuarto parámetro, el pH del sustrato. ¿Cuántos archivos modificas? ¿Por qué no menos?
- ¿En qué capa pondrías la gamificación y por qué no en el dominio de la planta? ¿O sí en el dominio?
- Señálame en tu código la línea donde un objeto del dominio deja de saber que existe HTTP.
- Si borro la carpeta de infraestructura, ¿compilan/pasan tus pruebas de dominio? Ejecútalas.
- Tu diagrama muestra este componente. Ábrelo en el repositorio. ¿Por qué tiene esta dependencia que el diagrama no muestra?
- ¿Qué decisión de tu documento tomarías distinto hoy, y qué te hizo cambiar de opinión?

El banco completo es más amplio que estos ejemplos, pero todas las preguntas son de esta naturaleza. Estudiar para la sustentación consiste en entender el propio diseño, no en memorizar definiciones.

8. Rúbrica

Componente	Peso	Criterio de nota máxima
Sustentación (preguntas de mutación)	35 %	Todos los integrantes localizan el impacto de los cambios en su propio código y justifican la frontera entre capas.
Cumplimiento de restricciones de arquitectura (verificable en el código)	25 %	RA1-RA8 satisfechas y demostrables. Cero acoplamiento del dominio con framework o persistencia.
Documento de arquitectura	25 %	Diagramas fieles al repositorio, justificación SOLID anclada a archivo y línea, plan de evolución coherente.
Front asíncrono y pruebas del dominio	15 %	Front funcional sin recarga, con manejo de errores; pruebas del dominio aisladas y verdes.
Bitácora de uso de IA (bonificación)	+5 %	Relato honesto y específico de qué aportó la IA, qué se corrigió y con qué criterio arquitectónico.

9. Uso de inteligencia artificial

El uso de asistentes de inteligencia artificial para construir este proyecto **está permitido y es esperado**. No hay que pedir permiso ni justificarlo. Saber trabajar con estas herramientas es parte del oficio, y este curso no va a fingir lo contrario.

Lo que sí importa es *cómo* se usan. Una IA puede producir en minutos un proyecto que parece bien arquitecturado y un

documento que suena convincente; si el equipo no entendió por qué está organizado así, ese resultado es una deuda, no un logro, y se va a notar la primera vez que el producto tenga que crecer —que en este proyecto es la próxima entrega, no un escenario lejano—. Por eso la evaluación está diseñada de modo que no se pueda aprobar delegando la comprensión: el 35 % se decide con el repositorio abierto, respondiendo qué cambia y qué no cambia ante una modificación que nadie preparó de antemano.

El documento de arquitectura debe ser redactado por el equipo a partir del análisis de su propio código. Reproducir una descripción generada a partir de este enunciado, sin que medie ese análisis, no es "usar IA": es entregar un texto que el equipo no puede defender, y la sustentación lo dejará en evidencia. Este documento incluye marcadores de trazabilidad que ayudan a identificar ese caso; cuando aparecen, abren una conversación con el equipo, no una sanción automática.

La bitácora suma, no resta. El entregable 6.e pide contar honestamente qué hizo la IA y qué hicieron ustedes. Declarar "le pedimos la estructura completa, nos propuso esto, lo cambiamos por esto otro porque acoplaba el dominio al framework" es exactamente la evidencia de aprendizaje que este corte busca, y se premia. Nadie pierde puntos por haber usado la herramienta; se ganan por entender lo que la herramienta produjo.

Anexo A — Contrato sugerido de la API

```
POST /api/v1/diagnosticos
Content-Type: application/json

{
  "especie": "sansevieria",
  "humedad": 32.5,
  "luz": 850,
  "temperatura": 21.0
}

200 OK
{
  "especie": "sansevieria",
  "estado": "EN_RIESGO",
  "parametros": [
    { "nombre": "humedad", "valor": 32.5, "unidad": "%", "rangoOptimo": [40, 70], "estado": "BAJO" },
    { "nombre": "luz", "valor": 850, "unidad": "lux", "rangoOptimo": [200, 1500], "estado": "OPTIMO" },
    { "nombre": "temperatura", "valor": 21.0, "unidad": "C", "rangoOptimo": [18, 29], "estado": "OPTIMO" }
  ],
  "recomendaciones": [
    "La humedad del sustrato esta por debajo del rango recomendado: riegue moderadamente."
  ]
}

404 NOT FOUND (especie desconocida)
{ "error": "ESPECIE_NO_SOPORTADA", "mensaje": "...", "detalle": { "especie": "..." } }

400 BAD REQUEST (parametro ausente, no numerico o fisicamente imposible)
{ "error": "PARAMETRO_INVALIDO", "mensaje": "...", "detalle": { "campo": "humedad" } }

GET /api/v1/especies
200 OK
[ { "nombre": "sansevieria", "rangos": { ... } }, ... ]
```

El contrato es una sugerencia razonable, no una imposición: el equipo puede modificarlo siempre que lo justifique en el documento y mantenga la uniformidad del cuerpo de error exigida por **RF6**.

Anexo B — Formato de la tabla de referencia

especie	humedad_min	humedad_max	luz_min	luz_max	temp_min	temp_max
sansevieria	20	45	200	1500	15	29
potos	40	70	300	1200	18	30
suculenta	10	30	800	2500	15	32
helecho	60	85	150	800	16	26
lavanda	25	50	1000	3000	15	30

Cinco especies es el mínimo. Si el equipo opta por base de datos en lugar de CSV, debe entregar el script de creación y carga. La elección entre CSV y base de datos **no debe ser visible desde el dominio**: esa es precisamente la comprobación de **RA5**.

Equipos: 3 o 4 integrantes. · **Entrega del repositorio y del documento:** __/__/__ · **Sustentaciones:** __/__/__ · La sustentación es individual dentro del equipo: la nota de este componente puede diferir entre integrantes.